

Tools:

- GitHub Copilot: <https://github.com/copilot> (create an account using your @info.uaic.ro email address)
- WSL: <https://learn.microsoft.com/en-us/windows/wsl/install> or a Linux distribution
- VirtualBox (optional): <https://www.virtualbox.org>
- Docker / Docker Desktop (optional): <https://www.docker.com>
- Visual Studio Code: <https://code.visualstudio.com/> (or any other IDE; eg. JetBrains)

To access the internal development server, please follow the instructions here:

https://docs.google.com/document/d/1L70rephVBh9r8_US4khX0_7zSalxcJJFJq31Nd86NCM

(Credits: Computer Networks Course, Conf. Dr. Andrei Panu)

You are not restricted to a specific programming language. We encourage diversity, so you can choose any programming language or IDE you are comfortable with. However, the lab examples will be presented using C/C++ and Python.

Ghid instalare WSL:

https://edu.info.uaic.ro/sisteme-de-operare/SO/lectures/Linux/Mini-ghid_instalare_WSL2.pdf

(user: so2026, pass: 20+26=46; **sau vezi linkul de mai sus de la WSL**)

Ghid instalare Linux Mint:

https://edu.info.uaic.ro/sisteme-de-operare/SO/lectures/Linux/Install_guide.pdf

(credits: Lect. Dr. Vidrascu Cristian, same user and pass)

Resources:

- pthreads: <https://man7.org/linux/man-pages/man7/pthreads.7.html>
- Google Sanitizers: <https://github.com/google/sanitizers>
- Linux udp(7) man page: <https://man7.org/linux/man-pages/man7/udp.7.html>
- Linux tcp(7) man page: <https://man7.org/linux/man-pages/man7/tcp.7.html>
- QUIC Working Group (IETF): <https://quicwg.org/>
- Cloudflare "The Road to QUIC" (accessible overview): <https://blog.cloudflare.com/the-road-to-quic/>
- quiche (Cloudflare, Rust/C): <https://github.com/cloudflare/quiche>
- ngtcp2 (C): <https://github.com/ngtcp2/ngtcp2>
- msquic (Microsoft, C): <https://github.com/microsoft/msquic>
- aioquic (Python): <https://github.com/aiortc/aioquic>
- RFC 9000 -- QUIC Transport Protocol: <https://www.rfc-editor.org/rfc/rfc9000>
- RFC 9001 -- Using TLS to Secure QUIC: <https://www.rfc-editor.org/rfc/rfc9001>
- RFC 9114 -- HTTP/3 (over QUIC): <https://www.rfc-editor.org/rfc/rfc9114>
- RFC 768 -- UDP specification (the entire spec is 3 pages): <https://www.rfc-editor.org/rfc/rfc768>
- RFC 9293 -- TCP specification (2022, replaces RFC 793): <https://www.rfc-editor.org/rfc/rfc9293>

- Beej's Guide to Network Programming (practical C examples):
<https://beej.us/guide/bgnet/>

LABORATORY 1: Concurrency, Nondeterminism, and Race Conditions

1. Laboratory Objectives

By the end of this laboratory, students will be able to:

1. **plain** the difference between the illusion of parallelism (time-slicing) and true parallelism.
2. **Empirically demonstrate** the nondeterministic nature of the OS process scheduler.
3. **Identify** and reproduce a Race Condition in a controlled environment.
4. **Utilize** mutual exclusion mechanisms (`pthread_mutex`) to protect critical sections.
5. **Apply** dynamic analysis tools (ThreadSanitizer) to detect invisible concurrency bugs.

Lab resources:

https://drive.google.com/drive/folders/1909FGpzGND8d5d5oG2QfyzwDIhUJymFJ?usp=drive_link

2. Theoretical Background

2.1. From Sequential to Concurrent

In a sequential program, execution is deterministic: Statement **A** is followed by Statement **B**. In concurrent programming, the order of execution between different threads is **undefined**. The Operating System Scheduler may interrupt a thread at any moment to run another.

2.2. Anatomy of a Race Condition

A Race Condition occurs when:

1. Two or more threads access the same memory location simultaneously.
2. At least one thread writes (modifies) that memory.
3. There is no synchronization mechanism to enforce an ordering.

Classic Example: The `count++` instruction.

Although it appears as a single line of C code, the CPU executes three distinct operations (Load-Modify-Store). If a Context Switch occurs between Load and Store, the update is lost.

2.3. The Critical Section and Mutex

The **Critical Section** is the segment of code that accesses a shared resource. This must be executed **atomically**.

A **Mutex** (Mutual Exclusion) acts as a lock/key:

- `lock()`: If the key is free, enter. If not, wait (block) until it becomes free.
- `unlock()`: Release the key for others.

3. Laboratory Activities

Prerequisites

- OS: Linux/macOS or Windows with WSL (Windows Subsystem for Linux).
- Compiler: `gcc`.

Exercise 1: The Illusion of Order

Goal: Demonstrate that the order of thread creation does not guarantee the order of execution.

1. Write a program that launches **30 threads**.
2. Pass a unique ID (0-29) to each thread upon creation.
3. The thread function should print: `[Thread ID] started execution`. and then `[Thread ID] finished`.
4. Run the program 5-10 times consecutively.
5. To make scheduling effects more visible, optionally synchronize thread start using a barrier (or run under CPU load).

Analysis Question:

Do you observe a guaranteed order (0 to 29)? Do the output lines appear mixed or interleaved?

Exercise 2: "The Broken Counter" - Race Condition

Goal: Observe data corruption. This is the core of the lab.

1. Create a global variable `volatile long counter = 0`;
2. Write a function `void* increment(void* arg)` that increments the variable **1,000,000 times** in a `for` loop.
3. In `main`, create **2 threads** that run this function simultaneously.
4. Wait for them to finish (`pthread_join`).
5. Print the final result.

Compilation: `gcc -o race race.c -pthread`

Analysis:

The expected value is 2,000,000. What value did you actually get?

Run `objdump -d race | grep -A 10 increment` (or similar). Identify the `mov`, `add`, `mov` instructions. Explain where the "window of vulnerability" lies.

Advanced Debugging (Google ThreadSanitizer):

Recompile the code using the sanitizer flag (standard in GCC/Clang):

- `gcc -o race_debug race.c -pthread -fsanitize=thread -g`
- `./race_debug`

Workaround (WSL/Kernel ASLR issue): If ThreadSanitizer fails at startup with `FATAL: ThreadSanitizer: unexpected memory mapping ...`, reduce ASLR entropy so TSan can reserve its shadow memory region:

```
sudo sysctl -w vm.mmap_rnd_bits=28
```

(Temporary setting—resets after reboot.)

Analyze the output. What type of error does TSan report? (e.g., `Data race on ...`).

Exercise 3: The Solution - Synchronization

Goal: Fix the bug using `pthread_mutex_t`.

1. Modify the code from Exercise 2.
2. Declare a global mutex: `pthread_mutex_t lock;`
3. Initialize it in `main` (`pthread_mutex_init`).
4. Protect **only** the incrementation of `counter` using `lock` and `unlock`.
5. Measure the execution time (using `time ./race` in the terminal) compared to the previous version.

Analysis Question:

Why is the program significantly slower now? Discuss *Context Switch Overhead* and *Serialization*.

4. Cheat Sheet: Local vs. Distributed Systems

Since this course lays the foundation for Distributed Systems, it is vital to understand the limitations of local concurrency models.

Feature	Concurrent Programming (Local)	Distributed Systems

Memory	Shared (Common Heap). Instant access.	No shared memory. Message passing only.
Synchronization	Mutex, Semaphores (OS Kernel).	Distributed Locks (e.g., Redis, ZooKeeper), Consensus (Raft/Paxos).
Time	Unique Global Clock (CPU Clock).	No global clock. (Clock Skew/Drift).
Failure	If the process crashes, everything stops.	Partial Failure: One node fails, others continue.
Latency	Nanoseconds.	Milliseconds (Network).

5. LabWork: "Bank Transfer & Deadlock"

Simulate a simple banking system with 2 accounts (A and B), each having its own Mutex.

1. **Thread 1** attempts to move money from A to B (Lock A -> Lock B).
 2. **Thread 2** attempts to move money from B to A (Lock B -> Lock A).
 3. Introduce a deliberate `sleep(1)` between the two lock acquisitions to force a **Deadlock**.
 4. **Fix:** Solve the problem by imposing a **Global Locking Order** (always lock the account with the smaller ID first).
-

LABORATORY 2: Network Communication & The Client-Server Model

1. Laboratory Objectives

By the end of this laboratory, students will be able to:

1. **Implement** a basic TCP client–server architecture using the Socket API.
2. **Differentiate** between an iterative (blocking) server and a concurrent (threaded) server.
3. **Observe** head-of-line blocking, where one slow client affects all other clients.
4. **Apply** thread creation techniques (from Lab 1) to handle network connections dynamically.
5. **Analyze** the fundamental trade-off between resource consumption (threads) and responsiveness (latency).
6. **Implement** a thread-safe task queue to support concurrency and handle multiple client/worker connections.
7. **Design** an application-layer protocol over TCP sockets.
8. **Compare** TCP, UDP, and QUIC in terms of reliability, ordering, congestion control, and latency.

Lab resources:

https://drive.google.com/drive/folders/175jG_WDyRPF3CaWz-XfMF-kaekbbGqH5?usp=drive_link

2. Theoretical Background

2.1. Message Passing vs. Shared Memory

In Lab 1, threads communicated by reading/writing to the same memory address. In Network Programming, processes (even on the same machine) do not share memory. They communicate by sending streams of bytes (Messages) through the OS kernel via **Sockets**.

2.2. The Socket API Lifecycle

A Socket is a file descriptor used for network I/O.

- **Server:** `socket()` -> `bind()` (reserve port) -> `listen()` (wait) -> `accept()` (handshake).
- **Client:** `socket()` -> `connect()` (initiate handshake).

2.3. Blocking I/O

Standard socket calls (`recv`, `accept`) are **blocking**.

- If the server calls `recv()` and the client hasn't sent data yet, the server **sleeps**.
- In a single-threaded (Iterative) server, while the server sleeps waiting for Client A, Client B cannot connect. This is the bottleneck we must solve.

2.4. TCP (Transmission Control Protocol)

A connection-oriented, reliable, stream-based protocol. Guarantees in-order delivery through sequence numbers, acknowledgments, and retransmission. Establishes connections via a 3-way handshake (SYN, SYN-ACK, ACK) and tears them down with a 4-way close (FIN/ACK). Provides flow control (sliding window) and congestion control (slow start, congestion avoidance).

Use when: reliability matters more than latency -- file transfers, web pages, database queries, the Coordinator-Producer-Worker system in this lab.

Example: HTTP/1.1, SSH, SMTP, PostgreSQL wire protocol.

```
// Minimal TCP client (C, POSIX)
int sock = socket(AF_INET, SOCK_STREAM, 0);
connect(sock, (struct sockaddr*)&addr, sizeof(addr)); // 3-way handshake
send(sock, data, len, 0); // reliable, ordered delivery
recv(sock, buf, buflen, 0); // stream-based (no message boundaries)
close(sock); // 4-way teardown
```

References:

- RFC 9293 -- TCP specification (2022, replaces RFC 793): <https://www.rfc-editor.org/rfc/rfc9293>
- Beej's Guide to Network Programming (practical C examples): <https://beej.us/guide/bgnet/>
- Linux tcp(7) man page: <https://man7.org/linux/man-pages/man7/tcp.7.html>

2.5 UDP (User Datagram Protocol)

A connectionless, unreliable, message-based protocol. No handshake, no acknowledgments, no ordering guarantees. Each `sendto()` maps to exactly one datagram; messages have clear boundaries (unlike TCP's byte stream). Minimal overhead (8-byte header vs. TCP's 20+ bytes).

Use when: low latency matters more than reliability -- real-time audio/video, DNS lookups, game state updates, IoT telemetry. Applications implement their own reliability if needed.

Example: DNS, DHCP, RTP (voice/video), game netcode.

```
// Minimal UDP sender (C, POSIX)
int sock = socket(AF_INET, SOCK_DGRAM, 0); // No connect() needed -- connectionless
sendto(sock, data, len, 0, (struct sockaddr*)&addr, sizeof(addr)); // fire and forget
recvfrom(sock, buf, buflen, 0, NULL, NULL); // one recv = one datagram
close(sock);
```


References:

- RFC 768 -- UDP specification (the entire spec is 3 pages): <https://www.rfc-editor.org/rfc/rfc768>
- Linux udp(7) man page: <https://man7.org/linux/man-pages/man7/udp.7.html>

2.6 QUIC (Quick UDP Internet Connections)

A modern transport protocol built on top of UDP, designed by Google and standardized by IETF in 2021. Combines the reliability of TCP with the low-latency connection setup of UDP. Key innovations:

0-RTT / 1-RTT connection setup: merges the transport handshake with TLS 1.3 encryption in a single round-trip (TCP+TLS requires 2-3 round-trips).

Stream multiplexing without head-of-line blocking: multiple independent streams over one connection. If one stream's packet is lost, other streams continue (unlike TCP, where a single lost packet stalls everything).

Connection migration: connections survive IP address changes (e.g., switching from Wi-Fi to cellular) because connections are identified by a Connection ID, not by IP:port tuple.

Built-in encryption: TLS 1.3 is mandatory; there is no unencrypted QUIC.

Use when: modern web applications, mobile-first apps, anything that needs multiplexed streams with encryption -- HTTP/3 runs entirely over QUIC.

Example: HTTP/3 (all major browsers), Google services, Cloudflare edge network.

- Note: QUIC is a userspace protocol (runs over UDP port 443), not a kernel-level protocol like TCP/UDP. It cannot be used with raw socket() calls -- it requires a library.

Libraries:

- quiche (Cloudflare, Rust/C): <https://github.com/cloudflare/quiche>
- ngtcp2 (C): <https://github.com/ngtcp2/ngtcp2>
- msquic (Microsoft, C): <https://github.com/microsoft/msquic>
- aioquic (Python): <https://github.com/aiortc/aioquic>

References:

- RFC 9000 -- QUIC Transport Protocol: <https://www.rfc-editor.org/rfc/rfc9000>
- RFC 9001 -- Using TLS to Secure QUIC: <https://www.rfc-editor.org/rfc/rfc9001>
- RFC 9114 -- HTTP/3 (over QUIC): <https://www.rfc-editor.org/rfc/rfc9114>
- QUIC Working Group (IETF): <https://quicwg.org/>

- Cloudflare "The Road to QUIC" (accessible overview):
<https://blog.cloudflare.com/the-road-to-quic/>

2. 7 Quick Comparison

	TCP	UDP	QUIC
Connection	3-way handshake	None	1-RTT (0-RTT for repeat)
Reliability	Yes (retransmit)	No	Yes (per-stream)
Ordering	Yes (global)	No	Yes (per-stream)
Encryption	Optional (TLS)	Optional (DTLS)	Mandatory (TLS 1.3)
Multiplexing	No (1 stream)	N/A	Yes (no HOL blocking)
Header size	20+ bytes	8 bytes	Variable (~16+ bytes)
Runs on	IP (kernel)	IP (kernel)	UDP (userspace)
Standard	RFC 9293	RFC 768	RFC 9000

3. Laboratory Activities

Exercise 1: The "Lazy" Iterative Server

Goal: Observe the limitations of a single-threaded server.

1. **Code (Provided in Appendix):** Compile the basic `server_iterative.c`. (the code is in Appendix)
2. **Action:** The server accepts a connection, reads a message, waits 5 seconds (simulating heavy processing), echoes the message back, and closes.
3. **Experiment:**
 - Open Terminal 1: Run Server.
 - Open Terminal 2: Run Client A (`nc localhost 8080`). Send "Hello".
 - *Immediately* Open Terminal 3: Run Client B (`nc localhost 8080`). Send "Hi".

Analysis Question:

What happens to Client B? Does it get a response immediately?

Why does Client B have to wait for Client A to finish, even though they are independent? (Explain "Head-of-Line Blocking").

Exercise 2: The Concurrent Server (Thread-per-Client)

Goal: Use concurrency to handle multiple clients simultaneously.

1. **Task:** Modify the `server_iterative.c` code to become `server_concurrent.c`.
2. **Logic:**
 - The `main` loop should only do `accept()`.
 - Immediately after accepting a connection (getting a `client_fd`), spawn a **new thread** (`pthread_create`).
 - The new thread executes the `handle_client` function.
 - **Important:** Don't forget to detach the thread (`pthread_detach`) to avoid memory leaks (zombie threads).

Analysis Question:

Run the same experiment as Ex 1 (Client A and Client B).

Does Client B still wait for Client A?

Thinking: What happens if 10,000 clients connect at the exact same second? What is the risk of "Thread-per-Client"? (Context Switch Overhead, Out of Memory).

Exercise 3: Simulating Network Unreliability

Goal: Understand that network writes are not instant.

1. Modify the Client to send a large amount of data (e.g., a 10MB file).
2. Introduce a `sleep(1)` inside the server's `recv/read` loop.
3. Observe that `send()` on the client side might return success *before* the server has actually processed the data.

Theory Check:

When `send()` returns, has the data reached the other application?

Answer: No. It has only been copied to the OS Kernel's TCP Send Buffer.

Objective: Demonstrate that when a generic `send()` function returns `Success`, it **does not** mean the data has reached the destination application. It only means the data has been copied to the Operating System's Kernel Buffer.

The Experiment:

1. **Fast Client:** Sends a large volume of data instantly and exits. (`client_fast.c`)
2. **Slow Server:** Reads data byte-by-byte with a delay. (`server_slow.c`)

How to Analyze

1. Run `./server_slow`.
2. Run `./client_fast`.
3. **Observation:** The Client says "Time taken: 0.000... seconds" and exits immediately. However, the Server will keep printing "Received chunk..." for almost a minute (due to `sleep(1)`).
4. **Conclusion:** The data was safely sitting in the **TCP Send Buffer** of the Client OS and the **TCP Receive Buffer** of the Server OS, not in the application.

4. LabWork: The Thread Pool Server

Context: Creating a thread for every client (Ex 2) is dangerous for scale. Real systems (like Nginx or Apache) use a **Thread Pool**.

Task:

1. Create a fixed pool of worker threads (e.g., 4 threads) at startup.
2. Implement a **Task Queue** (using a Linked List or Array).
3. The `main` thread accepts connections and adds the `client_socket` to the Queue.
4. Worker threads sleep until the Queue has an item (Producer-Consumer problem using Condition Variables - linking back to Lab 1).

Why this matters: This architecture prevents the server from crashing under load; it simply processes requests slower when the queue fills up, providing **Backpressure**.

Objective: Implement a production-ready server architecture using the **Producer-Consumer** pattern. Instead of creating a thread for every client (which crashes the OS at scale), we use a fixed number of threads consuming tasks from a queue.

Compilation & Testing

1. **Compile:**
 - a. Compile the Thread Pool Server: `gcc -o server_pool server_threadpool.c -pthread`
 - b. Compile this Stress Test Client: `gcc -o stress_test client_stress_test.c -pthread`
2. **Run the Experiment:**
 - a. **Terminal 1:** Start the server: `./server_pool`
 - b. **Terminal 2:** Run the stress test: `./stress_test`
3. **Expected Output & Analysis**
 - a. **On the Client Side (`./stress_test`):** You will see 20 lines appearing very quickly.
 - i. [Client 0] Server replied: Processed by ThreadPool

- ii. [Client 1] Server replied: Processed by ThreadPool
- iii. ...
- b. **On the Server Side (./server_pool)**: This is where the magic happens. Even though 20 clients arrived instantly, you will see the **Task Queue** logic in action:
 - i. You will see [Main] accepting connections rapidly and calling enqueue.
 - ii. You will see the 4 Worker Threads picking up tasks.
 - iii. **Key Observation**: Since there are only 4 threads, the logs will show them being reused.
 - 1. Example: [Worker Thread 12345] handles socket 5.
 - 2. Then [Worker Thread 12345] handles socket 9.
 - iv. If the clients were real users, the last few clients would experience a slightly higher latency than the first few, because they sat in the queue waiting for a worker to become free.

5. QUIC (msquic, quiche)

msquic

```
# --- SETUP INSTRUCTIONS ---
# 1. Install msquic runtime:
#   curl -sSL https://packages.microsoft.com/keys/microsoft.asc | sudo tee
/etc/apt/trusted.gpg.d/microsoft.asc
#   sudo apt-add-repository https://packages.microsoft.com/ubuntu/$(lsb_release -rs)/prod
#   sudo apt update
#   sudo apt install libmsquic
#
# 2. Build (headers are downloaded automatically):
#   make
#
# 3. Run (3 terminals):
#   Terminal 1: ./quic_coordinator
#   Terminal 2: ./quic_producer 5
#   Terminal 3: ./quic_worker
```

quiche

```
#
=====
# SETUP INSTRUCTIONS
#
=====
# 1. Install Rust (if not already):
#   curl --proto 'https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
#   source ~/.cargo/env
#
# sudo apt install build-essential cmake pkg-config libclang-dev libfontconfig1-dev
```

```
#
# 2. Build everything (clones quiche, compiles Rust lib, compiles C code):
#   make
#
# 3. Run (3 terminals):
#   Terminal 1: ./quiche_coordinator
#   Terminal 2: ./quiche_producer 5
#   Terminal 3: ./quiche_worker
#
# Notes:
# - First build takes 2-5 minutes (Rust compilation + BoringSSL).
# - Subsequent builds are fast (only C code recompiles).
# - `make distclean` removes the quiche-repo entirely.
# - The static library (libquiche.a) bundles BoringSSL, so no
#   separate OpenSSL/BoringSSL linkage is needed at runtime.
```

6. UDP

(see udp folder on Google Drive)

7. Coordinator-Producer-Worker

Coordinator (Broker): maintains a task queue, assigns tasks to ready workers, receives DONE reports.

Producer (Client): submits tasks (**SUBMIT** ...) from stdin or auto-generated.

Worker (Consumer): connects, requests work (**READY**), simulates processing (**sleep**), reports back (**DONE**).

https://drive.google.com/drive/folders/16QgmW7ImWIsHeGR_zc2DuhsFmYvnZgh1?usp=drive_link

LABORATORY 3: Distributed Systems Integrator – Architecture, Scalability & Security

1. Context & Objectives

In this laboratory, we will traverse the full lifecycle of a distributed system. We will not only write code that "works" but also analyze how it fails under pressure (scalability) and under malicious attacks (security).

Technical Objectives:

1. **Offensive Security (Red Team):** Exploit memory vulnerabilities (Buffer Overflow) using Python.
2. **Defensive Security (Blue Team):** Implement secure coding practices, Input Sanitization, and Challenge-Response Authentication.
3. **Infrastructure:** Simulate a real network (**you may use VMs, Dockers**).
4. **Extensions:** Create additional extensions for your Client-Coordinator-Worker infrastructure.

Laboratory Objectives:

- **(Build):** Implement the functional (but naive) Coordinator-Worker system.
- **(Break):** The "Red Team" session. Crash the system using exploits.
- **(Fix):** The "Blue Team" session. Hardening, Authentication, and Fault Tolerance.
- **(Enhance Functionality):** Implement extension
- **Discuss Homework**

Lab resources:

https://drive.google.com/drive/folders/1xaINveOR0hdol1eZmZsvgZ486NOd-oFh?usp=drive_link

PART I: ARCHITECTURE & THREAT MODELING (Theory)

1.1. The System Architecture

We will implement a **Distributed Producer-Consumer** model.

- **Client (Producer):** Generates tasks (e.g., "Process Image #ID") and submits them.
- **Coordinator (Broker):** The central node. It manages a Task Queue and assigns tasks to available Workers.
- **Worker (Consumer):** Connects to the Coordinator, requests work, executes it (simulated delay), and reports back.
- TODO: select / poll

1.2. Threat Modeling

Before coding, we analyze the risks. A distributed system written in C listening on a public port is a prime target.

1. **Spoofing:** An attacker pretends to be a Worker to steal data or corrupt results. -> *Solution: Authentication.*
2. **Tampering (Buffer Overflow):** An attacker sends a packet larger than the allocated buffer to overwrite server memory. -> *Solution: Bounds Checking.*
3. **Denial of Service (DoS):** Exhausting server threads or memory. -> *Solution: Thread Pools / Timeouts.*

PART II: THE "NAIVE" IMPLEMENTATION

Goal: Get it working. Ignore security for now.

Step 1: The Common Protocol (`common.h`) (RedTeam folder)

Step 2: The Vulnerable Coordinator (`coordinator_naive.c`) (RedTeam Folder)

This server uses a Mutex for the queue (Concurrency) but uses `strcpy` (Security Flaw). (Compile: `gcc coordinator_naive.c -o coord_naive -pthread`)

PART III: RED TEAM EXERCISE

Goal: Crash the system without touching the C code.

The Exploit: We will write a Python script that sends a packet with a description larger than 64 bytes. Since `coordinator_naive.c` uses `strcpy`, this will cause a segmentation fault.

The Attack Script (`exploit.py`) (RedTeam Folder)

Note on Endianness: We are using `struct.pack("iii")` which uses the machine's native byte order (Little Endian on Intel). In a real production protocol, you must use Network Byte Order (Big Endian).

- Python: `struct.pack("!iii", ...)`
- C Code: Use `htonl()` when sending and `ntohl()` when receiving integers.

Activity:

1. Run `./coord_naive`.
 2. Run `python3 exploit.py`.
 3. **Result:** The Coordinator crashes (`Segmentation fault`).
-

PART IV: BLUE TEAM - FIX & SECURE

Goal: Rebuild the Coordinator to be secure and robust.

Improvements Required:

1. **Secure Coding:** Replace `strcpy` with `strncpy` and manually enforce NULL termination. Check `recv` return size.
2. **Authentication:** Implement a Challenge-Response handshake to prevent unauthorized workers.
3. **Scalability:** Full implementation of the Mutex Queue and Worker logic.

The Secure Coordinator (`coordinator_secure.c`) (BlueTeam_and_AdvancedSimulation Folder)

The Authenticated Worker (`worker_secure.c`) (BlueTeam_and_AdvancedSimulation Folder)

(You also need a simple `client.c` which generates random tasks, similar to previous labs).

File: `client.c` (The Task Producer) (BlueTeam_and_AdvancedSimulation Folder)

Compilation & Usage

Compile: `gcc client.c -o client`

Usage: `./client`

ARCHITECTURAL NOTE: Why Authenticate Only Workers?

In this laboratory, we implement a "**Public Submission / Trusted Processing**" model.

- **Unauthenticated Clients:** Simulates a public endpoint (like an IoT sensor or web form) where ease of submission is prioritized over access control.
- **Authenticated Workers:** Simulates the "Trusted Zone". Since Workers receive data to process, a rogue Worker could steal sensitive information or corrupt results. Therefore, **Worker Authentication** is the critical security boundary we must enforce to prevent Data Exfiltration.

In a production environment, you would likely implement Authentication (API Keys or JWT) for Clients as well to prevent Denial of Service (DoS) attacks.

PART V: ADVANCED SIMULATION - RESOURCE EXHAUSTION (DoS)

Goal: Demonstrate how algorithmic inefficiency and lack of back-pressure mechanisms lead to system collapse under load.

Theoretical Context: In a blocking I/O model (like our current Coordinator), the `accept()` system call removes a connection from the OS **Backlog Queue**. If the application thread cannot call `accept()` fast enough (because it is busy spawning threads or waiting on a Mutex), the OS queue fills up. Once full, the OS kernel drops new SYN packets, and legitimate clients experience a "Connection Timed Out".

1. The Malicious Tool (`dos_flooder.c`) (BlueTeam_and_AdvancedSimulation Folder)

Create this file on the **Attacker Machine**. It uses threads to open hundreds of connections per second, flooding the Coordinator's `accept` queue and `mutex`.

Compile: `gcc dos_flooder.c -o flooder -pthread`

2. The Experiment Setup

Phase A: Establishing Baselines

Monitor the Coordinator: Open a terminal to stream logs: `docker compose logs -f coordinator`

Monitor Resources: In a separate terminal, use: `docker stats coordinator`

- *Observe the CPU limit. It should not exceed 50% (as configured in YAML).*

Run a Baseline Test: `./client "Baseline Task"`

- *Result: Immediate ACK. Processing time matches the random duration.*

Phase B: The SYN/Connection Flood We will now saturate the Coordinator's ability to accept connections.

Launch the Attack: `./flooder &`

Analyze the Degradation (Quantitative Analysis): While the attack is running, attempt to run a legitimate client: `/client "Critical Payload"`

- **Metric 1 - Latency:** Compare the `real` time output. It should increase from `<0.1s` to multiple seconds or timeout completely.
- **Metric 2 - Queue Saturation:** Inspect the Coordinator's internal TCP state: `netstat -ant | grep :9000 | grep ESTABLISHED | wc -l`
- *This counts how many connections are currently open.*

Kernel-Level Inspection: Verify the backlog overflow. Run this inside the Coordinator container: `ss -lnt`

- Look at the `Recv-Q` column for port 9000.
 - **Send-Q:** The max backlog size (likely 128 or SOMAXCONN).
 - **Recv-Q:** The current pending connections.
 - *Observation:* During the attack, `Recv-Q` will equal `Send-Q` (e.g., 128/128). This proves the kernel is rejecting new connections.

PART VI: SYSTEM EXTENSIONS

(Reliability & Defense) - *SystemExtensions* Folder

Objective: Implement mechanisms to handle failures (Fault Tolerance) and attacks (DoS Mitigation).

EXTENSION A: FAULT TOLERANCE (Retry Mechanism)

The Problem: In the current secure coordinator, if a Worker crashes *after* taking a task but *before* finishing it, that task is lost forever (dropped from the queue). **The Solution:** Implement the "At-Least-Once Delivery" pattern.

1. When a task is dequeued, move it to a `Pending List` with a timestamp.
2. A background thread (Monitor) checks this list every second.
3. If `CurrentTime - StartTime > TASK_TIMEOUT`, assume the worker died and move the task back to the main Queue.

EXTENSION B: DoS MITIGATION (Rate Limiting)

The Problem: The `dos_flooder` tool proved that a single IP can exhaust our resources. **The Solution:** Implement a simple **Token Bucket** or **Request Counter** per IP. If an IP exceeds 5 requests/second, drop the connection immediately.

EXTENSION C: GRACEFUL SHUTDOWN (Signal Handling)

The Problem: Using `Ctrl+C` kills the server instantly, potentially corrupting the queue or leaving sockets in `TIME_WAIT`. **The Solution:** Handle `SIGINT`.

EXTENSION D: SMART SCHEDULING (Priority Queues)

The Problem: Currently, the system uses a **FIFO** (First-In, First-Out) model. If a client submits a low-priority "Batch Job" that takes 1 hour, and immediately after another client submits a "Critical User Login" task, the critical task waits behind the slow one. **The Solution:** Implement a **Priority Queue**.

1. Modify the protocol to accept a `priority` level (1=Low, 10=Critical).
2. Update the `enqueue` logic to place High Priority tasks at the front of the line.

EXTENSION E: RESOURCE MANAGEMENT (Cluster Monitoring Dashboard)

The Problem: The Coordinator is a "black box". We don't know how many workers are connected, who is idle, or who is working on what. **The Solution:** Implement a **Worker Registry** and a Dashboard Thread that prints the cluster state every 5 seconds.

PCD | Homework 1

Write a program that measures the time required to transfer various amounts of data (500 MB, 1 GB) using different transport protocols and different transfer methods. The program should include detailed measurements to analyze the performance of distributed networks using **TCP**, **UDP**, and **QUIC** protocols.

Implementation Requirements:

1. Supported Protocols (as parameters for client and server):

- **TCP** (Transport Control Protocol):
 - Provides reliable connections with flow control and in-order packet delivery.
- **UDP** (User Datagram Protocol):
 - Provides fast transport without flow control or error handling mechanisms.
- **QUIC** (Quick UDP Internet Connections):
 - Provides fast, reliable, and secure connections with built-in multiplexing, avoiding head-of-line blocking issues.
 - You can use one of the following libraries: **ngtcp2**, **quiche**, **lsquic**, **msquic**, et.al.

2. Message Size:

- Transfer data in blocks with sizes ranging from **1** to **65535 bytes**.
- Test at least 5 different block sizes within this range for each protocol (TCP, UDP, QUIC). Example sizes: 1; 500; 1,000; 10,000; 65,535 bytes.

3. Data Transfer Amounts:

- Transfer **500 MB** and **1 GB** of data.
- You may use a large reusable buffer or simulate real-world data structures (e.g., "1 GB = 1.5 million WhatsApp messages, 4000 photos, or 10,000 emails").

4. Hashing Function

- Both the client and the server must implement a hashing mechanism over the transferred data to validate the correctness and integrity of the implementation.
- **For example:**
 - The client computes a SHA-256 hash of the entire data payload (e.g., the generated buffer or input file) before sending it, and prints the resulting hash.
 - The server reconstructs the received data (in the correct order, as applicable), computes a SHA-256 hash over the assembled payload, and prints the resulting hash.
- **The hashes printed by the client and server must match for a successful transfer.**

5. Transfer Mechanisms:

- **Streaming:**
 - Continuous transfer of data without waiting for confirmation after each message.
- **Stop-and-Wait:**
 - Acknowledgment for each message before sending the next one.
 - Implementing **stop-and-wait** for **TCP** is optional since TCP already manages acknowledgments and retransmissions automatically.
 - Implementing **stop-and-wait** for **UDP** and **QUIC** is mandatory to analyze its impact on performance and data loss.

Output Requirements:

1. **The Server** must display the following at the end of each session:
 - The protocol used: **TCP**, **UDP**, or **QUIC**.
 - The total number of messages received.
 - The total number of bytes received.
2. **The Client** must display the following at the end of the execution:
 - Total transmission time (in seconds).
 - The total number of messages sent.
 - The total number of bytes sent.
3. **Testing and Report:**
 - Students must perform comparative tests using all three protocols (TCP, UDP, QUIC) for various message sizes and data amounts (500 MB and 1 GB).
 - The analysis must include detailed comparisons of:
 - Transmission time for each protocol.
 - Data loss rates (for UDP and QUIC).
 - The impact of message size on performance.

Evaluation Criteria:

1. Complete, functional client and server code that runs on a **Linux** system.
2. A PDF (Word/LaTeX) document containing:
 - Detailed information on how to use the program (options for client/server).
 - Statistics and comparisons of performance for each protocol.
 - A detailed analysis of the experimental results.
 - Conclusions about the advantages and disadvantages of each protocol under different transfer conditions.

General Observations:

1. If the buffer size for **UDP** is increased, data loss may be minimal in a **LAN**, but packet loss may occur in more complex networks.

2. The **stop-and-wait** mechanism is not very useful for **TCP**, as TCP already handles acknowledgments, but it may be interesting to analyze its impact.
3. The **stop-and-wait** mechanism is more relevant for **UDP** and **QUIC**, especially in contexts where message loss needs to be minimized.

Instrucțiuni de trimitere a temei

Format arhivă:

- **Nume fișier:** Tema1_NumePrenume_2026_AcronimMaster.zip (fără diacritice)
- **Exemplu:** Tema1_PopescuIon_2026_MSD.zip
- **Conținut:**
 - Cod sursă - propriu (fără librării)
 - Documentul PDF rezultat din Word sau LaTeX
- **Locație:** URL-ul indicat în formularul de înregistrare (puteți modifica link-ul trimis doar dacă este absolut necesar).

PART V: DEPLOYMENT - CONTAINERIZED ARCHITECTURE

Goal: formalize the distributed system environment using Docker, moving from ad-hoc process management to an orchestrated microservices architecture.

1. Architectural Justification

Instead of VMs, we utilize **OS-level virtualization (Containers)**. This allows us to:

1. **Deterministic Environment:** Guarantee that the memory layout and network stack are identical for all students, ensuring exploits (like Buffer Overflows) behave consistently.
2. **Resource Quotas:** We will artificially constrain the Coordinator's CPU resources. This is critical for the **DoS simulation**, as it prevents your powerful host machine from masking the inefficiency of the naive server implementation.
3. **Network Isolation:** We will define a software-defined bridge network (`172.25.0.0/16`) to isolate experiment traffic from the host.

2. Infrastructure Setup

Step 1: Project Structure Organize your workspace to separate source code from deployment logic.

```
/lab3_dist_systems
├── src/
│   ├── coordinator_secure.c
│   ├── worker_secure.c
│   ├── client.c
│   ├── dos_flooder.c (see Part VI)
│   └── common.h
├── Dockerfile
└── docker-compose.yml
```

Step 2: The Multi-Stage Dockerfile Create a **Dockerfile** that uses a **multi-stage build**. This is an industry standard to keep runtime images small and secure. We compile in a heavy "builder" stage and copy only the binaries to a lightweight "runtime" stage.

```
# --- Stage 1: Builder ---
FROM gcc:12 AS builder
WORKDIR /build
```

```

# Copy source code
COPY src/ .

# Compile binaries with debug symbols (-g) and pthread support
# We compile specific targets to avoid rebuilding everything unnecessarily
RUN gcc coordinator_secure.c -o coord -pthread -g
RUN gcc worker_secure.c -o worker -pthread -g
RUN gcc client.c -o client -g
RUN gcc dos_flooder.c -o flooder -pthread -g

# --- Stage 2: Runtime Environment ---
# We use a standard Debian-slim image for a realistic server environment
FROM debian:bookworm-slim

# Install networking tools for "Blue Team" analysis
# netcat: for manual socket interaction
# tcpdump: for packet capture
# iproute2: for 'ss' and 'ip' commands
# procps: for 'ps' and 'top'
RUN apt-get update && apt-get install -y \
    netcat-openbsd \
    tcpdump \
    iproute2 \
    procps \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

# Copy only the compiled binaries from the builder stage
COPY --from=builder /build/coord .
COPY --from=builder /build/worker .
COPY --from=builder /build/client .
COPY --from=builder /build/flooder .

# Default entrypoint (overridden by Compose)
CMD ["/coord"]

```

Step 3: Orchestration (docker-compose.yml) Create `docker-compose.yml`. Note the explicit **subnet configuration** and **resource limits**.

```

version: '3.8'
services:

```

```
coordinator:
  build: .
  container_name: coordinator
  command: ./coord
  ports:
    - "9000:9000"
  networks:
    backend_net:
      ipv4_address: 172.25.0.10 # Static IP for legacy clients
  deploy:
    resources:
      limits:
        cpus: '0.5'
        memory: 256M
  sysctls:
    - net.ipv4.tcp_syncookies=1 # Mitigation for SYN Flood

worker:
  build: .
  command: ./worker
  deploy:
    replicas: 3
  networks:
    - backend_net
  depends_on:
    - coordinator

attacker:
  build: .
  container_name: attacker
  entrypoint: ["tail", "-f", "/dev/null"]
  networks:
    - backend_net

# DEFINE THE NETWORK
networks:
  backend_net:
    driver: bridge
    ipam:
      config:
        - subnet: 172.25.0.0/16
```

3. Code Adaptation

You must update the addressing in `src/common.h` to match the static IP defined in the Compose file.

```
// src/common.h
// #define SERVER_IP "127.0.0.1"    <-- Legacy
#define SERVER_IP "172.25.0.10"    // <-- Docker Static IP
```

Workflow:

1. **Build:** `docker compose build`
2. **Deploy:** `docker compose up -d`
3. **Verify:** `docker compose ps` (Check that 5 containers are Up).

4. Network Observability & Protocol Analysis

Unlike VirtualBox Host-Only networks (where the traffic passes through a virtual adapter visible to the host OS), Docker containers on non-Linux systems (Windows/Mac) run inside a lightweight utility VM. This makes "sniffing" the traffic from your Host Wireshark slightly more complex.

We will use two methods. Choose the one suitable for your OS.

Method A: The "Sidecar" Capture (Universal - Recommended for Windows/Mac) Since we cannot easily bridge the Host Wireshark to the internal Docker network on Windows/Mac, we will use `tcpdump` *inside* the Coordinator container to capture traffic to a file, and then analyze it on the host.

Start the Capture: Exec into the running coordinator and start a background capture: `docker exec -d coordinator tcpdump -i eth0 port 9000 -w /app/capture.pcap:`

- `-i eth0`: The container's internal interface.
- `-w ...`: Write to a file instead of printing to stdout.

Generate Traffic: Run your `client` or `worker` interactions.

- *Example:* `docker exec attacker ./client "Packet Capture Test"`

Stop & Extract: Since `tcpdump` is running in the background, we simply copy the file out (the process will die when the container stops, or you can kill it): `docker cp coordinator:/app/capture.pcap ./trace.pcap`

Analysis: Open `trace.pcap` in Wireshark on your host machine.

Method B: Direct Bridge Analysis (Linux Native Only) If you are running Docker natively on Linux, the Docker network appears as a real bridge interface on your host.

Identify the Interface: `ip addr | grep br-`

1. Look for the interface associated with `172.25.x.x`. It will likely be named something like `br-0a1b2c3d4e5f`.
2. **Live Capture:** Open Wireshark, select that bridge interface, and apply the display filter: `tcp.port == 9000`

Analysis Objectives (What to look for):

Once you have the packets in Wireshark (via Method A or B), locate the following frames to verify your binary protocol implementation:

1. **The Handshake (Authentication):**
 - Find the `MSG_WORKER_READY` packet (Type 2).
 - Immediately following it, look for the **Server Challenge** (4 bytes, the `Nonce`).
 - Look for the **Worker Response** (4 bytes, `Nonce ^ Secret`).
 - If you see the Nonce and the Response in plaintext, what does this imply about Man-in-the-Middle (MitM) attacks? (Answer: The protocol is vulnerable to replay attacks or offline derivation of the key if many pairs are captured).
2. **The Payload (Buffer Overflow Context):**
 - Find a `MSG_TASK_SUBMIT` packet.
 - Inspect the "Data" pane in Wireshark. You will see the raw struct bytes.
 - Verify the byte alignment. Is the compiler padding the struct? (This is why `__attribute__((packed))` is often discussed in network programming).
3. **The DoS Artifacts (During Part VI):**
 - If you capture during the DoS attack, you should see thousands of **SYN** packets from `172.25.0.20` followed by **SYN**, **ACK** from the Coordinator, but *missing* the final **ACK** (if using a SYN flood tool) or simply fully established connections that transmit garbage data.

Attack simulation:

1. Exec into the `attacker` container and run a legitimate client: `docker exec -it attacker ./client "Baseline Task"`
2. Inside the attacker container, run the flooder tool: `docker exec -d attacker ./flooder`
 - a. (The `-d` runs the exec command in the background).

3. `docker exec -it attacker ./client "Critical Payload"`
4. `docker exec coordinator netstat -ant | grep :9000 | grep ESTABLISHED | wc -l`
5. `docker exec coordinator ss -lnt`

Advanced Mitigation (Blue Team Implementation)

For the mitigation phase (Part VI, Extension B), you are expected to implement application-layer defense. However, a robust defense requires **Kernel Tuning** as well.

Task: Modify the `docker-compose.yml` to harden the kernel parameters for the Coordinator.

```
coordinator:
  # ... previous settings ...
  sysctls:
    - net.core.somaxconn=1024    # Increase max backlog queue
    - net.ipv4.tcp_max_syn_backlog=2048
```

Note: This strictly buys you time; it does not solve the root cause. The true solution requires the Rate Limiting logic implemented in C (Extension B).

Cleanup & Reset

Between experiments, you must fully tear down the stack to clear the orchestrator state:

```
docker compose down
```